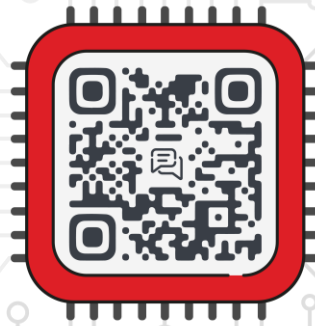




4



9



1080 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Inner classes

عملي مشترك



RB Informatics; 14/11/2024

برمجة ٣

Inner classes:

متى نستخدمها بشكل عام :

عندما لا يكون من المنطقي وجود object من class ما خارج إطار class آخر ضمن برنامجنا ، أي يوجد تبعية منطقية بين two classes .

أي مثلاً لا نريد وجود عجلة دون سيارة، فنعرف صف العجلة ضمن السيارة مما يجعله تابعاً للسيارة وحسب، ولا يمكن الوصول له من خارج السيارة و عمل object مستقل منه. يتميز أنه يستطيع الوصول لكافة بيانات الكلاس الخارجي الذي يحويه.

- لهذا الصف الداخلي عدة أنواع تدعم نفس الغرض:

static inner class - non-static inner class - functional inner class - anonymous class

جميعهم يخدمون نفس الغرض بطرق مختلفة، لن ندخل بالتفصيل كون المحاضرة لا تتطرق لتمرارين عملية على جميعها.

Non static inner class :

يتبع لا object من class الخارجي، ويتطلب عمل كائن من الصف الخارجي حتى نستطيع الوصول للصف الداخلي وعمل كائن منه.

Static inner class :

نستطيع الوصول له بدون عمل object من الصف الخارجي بشكل مباشر ، و ليس متعلقاً بأي كائن إنما متعلق بالclass الخارجي بحد ذاته (يمكن عمل object منه من خلال ال class الخارجي بدون object)

قم بعمل PC Class :

attribute : price

with two inner classes:

■ Processor :

attributes : manufacturer (اسم المصنِّع) , cores (عدد الأنوية)

methods : getCache (ترجع حجم ذاكرة الكاش)

■ RAM :

attributes : manufacturer (اسم المصنِّع) , memory (سعة الذاكرة)

methods : getSpeed

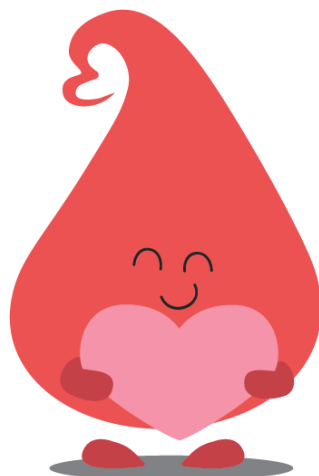
بعدها عرف:

Main Class:

and main method

Define PC object then Processor and RAM objects

Then print the speed of the memory and the cache of the processor.



```

1 //Class Pc
2 public class Pc {
3     double price;
4
5     //Inner Class Processor
6     class Processor {
7         double cores;
8         String manufacturer;
9
10        double getCache() {
11            return 4.3;
12        }
13    }
14
15    //Inner Class RAM
16    class RAM {
17        double memory;
18        String manufacturer;
19
20        double getSpeed() {
21            return 5.5;
22        }
23    }
24 }
25

```

```

1 public class Main {
2     public static void main(String[] args) {
3         // Pc object
4         Pc pc = new Pc();
5         //instantiating object from the
        Processor class using object of the outer
6         Pc.Processor processor = pc.new
        Processor();
7         //instantiating object from the RAM
        class using object of the outer
8         Pc.RAM ram = pc.new RAM();
9         //PRINT INFO OF RAM , PROCESSOR
        التابعين للـ pc
10        System.out.println("Processor Cache
        = " + processor.getCache());
11        System.out.println("Ram Clock speed
        = " + ram.getSpeed());
12    }
13 }

```

قم بتعريف:

Car class with:

- Attributes: carName, carType
- Parameterized constructor
- Inner class: **Engine** , that has:
 - Attribute: EngineType.
 - Method: setEngine (Defines the engine as (Bigger) if the car is 4WD, and as (Smaller) otherwise).

Main class with: main method that defines Car object and Engine object, and uses setEngine() then prints the engine type.

```
1 //Car Class
2 public class Car {
3     //attributes
4     String name;
5     String type;
6     //constructor
7     Car(String name , String type){
8         this.name = name;
9         this.type =type;
10    }
11    //inner class Engine
12    class Engine{
13        private String engineType ;
14        void setEngineType(){
15            if(Car.this.type.equals("4WD"))
16                this.engineType = "bigger";
17            else
18                this.engineType = "smaller";
19        }
20    }
21    public String getEngineType(){
22        return engineType;
23    }
24
25 }
26 }
```

```
1 public class Main {
2     public static void main(String[] args) {
3         //object of Car
4         Car car = new Car("Mazda", "8WD");
5         //object of engine follows car
6         Car.Engine engine = car.new
7 Engine();
8         //it would give the engine type value of
9 smaller
10        engine.setEngineType();
11        //printing the engineType
12        System.out.println("engine type is
13 : "+engine.getEngineType());
14    }
15 }
```

عرف:

- interface with getTotalPorts method
- Class MotherBoard that has USB as static inner class
in USB we have usb2 and usb3
تمثل عدد مأخذ الusb
من كلا النوعين
override getTotalPorts and make it return the sum of the usb ports
- then in Main class make main method and make object of usb and print the total ports

```
1 //Port Interface
2 public interface Port {
3     int getTotalPorts();
4 }
```

```
1 //MotherBoard Class
2 public class MotherBoard {
3     //static inner class USB implements
    the Port interface
4     static class USB implements Port
5     {
6         int usb2 = 3;
7         int usb3 = 5;
8         @Override
9         public int getTotalPorts() {
10             return usb2+usb3;
11         }
12     }
13 }
```

```
1 //Main Class
2 public class Main {
3     public static void main(String[] args){
4         //making an object of the inner
        class directly "its static"
5         MotherBoard.USB usb = new
        MotherBoard.USB();
6         System.out.println("Total Ports = "
        + usb.getTotalPorts());
7     }
8 }
```

نلاحظ أننا بسبب قيامنا بتعريف كلاس USB كstatic استطعنا الوصول إليه وإنشاء object منه دون إنشاء object من الكلاس الخارجي (MotherBoard).

التمرين غير المحلول :

■ عرف Robot class فيه two inner classes:

Brain class:

يحتوي method think() الذي يطبع عبارة "robot is thinking"

Battery class:

يحتوي method charge() الذي يطبع عبارة "robot is charging"

و في Robot لدينا start method تعرف object من Brain و تستدعي التابع think

■ و في التابع الرئيسي ضمن Main class نقوم بإنشاء object من Robot ونستدعي التابع start ، ونقوم بإنشاء object من Battery و نستدعي التابع charge

```

1 //Robot class
2 public class Robot {
3     //Brain class
4     class Brain {
5         public void think() {
6             System.out.println("the Robot
7 is thinking");
8         }
9     }
10
11     //Battery class
12     class Battery {
13         public void charge() {
14             System.out.println("the Robot
15 is charging");
16         }
17     }
18
19     //start method
20     public void start() {
21         //new object of brain then use
22         think method
23         new Brain().think();
24     }

```

```

1 public class Main {
2     public static void main(String[] args){
3         //robot object first
4         Robot robot = new Robot();
5         //start method call
6         robot.start();
7         //battery object follows the robot object
8         Robot.Battery battery = robot.new
9 Battery();
10        //charge method call
11        battery.charge();
12
13    }
14 }

```

قم بعمل:

CalculateAverage interface that has : getAverage method

وقم بعمل:

Student class that has:

- attributes : name , list of grades
- parameterized constructor and two inner classes
- Avg class implements CalculateAverage
overrides the getAverage method
لحساب متوسط علامات الطالب
- maxDegree class
with method getMax
ترجع أعلى علامة لدى الطالب

then in main method in Main class

make the user enter some students with their info

then make the user choose index of one of them then print the student average of grades and the max degree of that certain student.

تذكر دائماً أن أكبر الانتصارات تأتي بعد أصعب المحن

```

1 //CalculateAverage interface
2 public interface CalculateAverage {
3     public double getAverage();
4 }

1 import java.util.ArrayList;
2 //Student Class
3 public class Student {
4     //attributes
5     private String name;
6     private ArrayList<Double> grades;
7
8     //constructor
9     Student(String name, ArrayList<Double>
grades) {
10
11         this.name = name;
12         this.grades = grades;
13     }
14     //Avg inner class
15     class Avg implements CalculateAverage
{
16         @Override
17         public double getAverage() {
18             // نحصل على المجموع لجميع علامات
الطالب ضمن grades arraylist ثم نقسمها على حجم
arraylist
19             double sum = 0;
20             for (int i = 0; i < grades.size(); i++)
21                 sum += grades.get(i);
22             return sum / grades.size();
23         }
24     }
36 //MaxDegree class
37 class MaxDegree {
38     public double getMax() {
39         // نضع max بقيمة سالبة (أصغر من أصغر
علامة ممكنة) ثم نستمر بالمرور على كافة عناصر arraylist
ونجعل max تساوي القيمة الأكبر بالمقارنة بين max والعلامة
التي نقف عندها حتى ننتهي من كل عناصر arraylist

```

```

40         double max = -1;
41         for (int i = 0; i < grades.size();
i++)
42             max = Math.max(max,
grades.get(i));
43         return max;
44     }
45 }
46
47 }

```

```

1 import java.util.ArrayList;
2 import java.util.Scanner;
3
4 //Main Class
5 public class Main {
6     public static void main(String[] args) {
7         //arraylist of students
8         ArrayList<Student> arrayList = new
ArrayList<>();
9         Scanner scanner = new
Scanner(System.in);
10         System.out.println("number of
students");
11         int number = scanner.nextInt();
12         scanner.nextLine();
13         for (int i = 0; i < number; i++) {
14             System.out.println("enter the
name of the student");
15             String name = scanner.nextLine();
16             System.out.println("enter the
grades of the student");
17             //grades list
18             ArrayList<Double> grades = new
ArrayList<>();
19             for (int j = 0; j < 4; j++) {
20                 System.out.println("enter the
" + (j + 1) + " th " + " grade");
21                 grades.add(scanner.nextDouble()); }

```

```
22 scanner.nextLine();
23 //making new student
24 Student student = new Student(name, grades);
25 //add him to the list
26 arrayList.add(student);
27 }
28 System.out.println("choose the index of the student you want to get average and
the max degree for");
29 int ser = scanner.nextInt();
30 System.out.println("the average of his grades is");
31 //نختار الطالب ضمن الـ arrayList الذي له الـ ser index ثم نحضر متوسط علاماته هنا
32 //نعمل object من Avg تابع للـ student المحدد
33 Student.Avg avg = arrayList.get(ser).new Avg();
34 System.out.println(avg.getAverage());
35 System.out.println("the max grade of him is");
36 //وأعلى علامة له هنا
37 //ننشئ object من MaxDegree تابع للـ student المحدد كذلك
38 Student.MaxDegree maxDegree = arrayList.get(ser).new MaxDegree();
39 System.out.println(maxDegree.getMax());
40
41 }
42 }
```

The End